CSE 4101: Compiler Design

Assignment: “Sci-PL” — A Mini Compiler for Scientific Computation

Objective

Design and implement a simple compiler for a small programming language called Sci-PL. The language supports basic arithmetic, control flow, and one special scientific feature for checking chemical-reaction balance.

You will apply the key phases of compiler design — lexical analysis, syntax analysis, semantic analysis, and code generation — in one integrated project.

Language Overview

Data Types: int, float, bool, reaction

Example:

```
def int x;
def float y;
def reaction r1;

x = 10;
y = x + 5.5;
r1 = "2*H2 + O2 -> 2*H2O";
```

Operators:

- Arithmetic: +, -, *, /
- Relational/Logical: >, <, ==, &&, ||
- Control: if (...) {…} else {…}, while (...) {…}
- Automatic type promotion: int → float

Special Operator:

check_balance(expr) → returns true if the chemical equation is balanced.

Example:

```
bool b = check_balance(r1);
if (check_balance("H2 + O2 -> H2O")) { ... }
```

All programs start with:

```
main { ... st atements ... }
```

Source Code

```
main {
  def int x;
  def float y;
  def bool isBalanced;
  def reaction r1, r2;
  x = 10;
  y = x + 5.5;
  r1 = "2*H2 + O2 -> 2*H2O";
  r2 = "H2 + O2 -> H2O";
  isBalanced = check_balance(r1);
  if (check_balance(r2)) {
      x = x + 1;
  } else {
      x = x - 1;
  }
  while (x > 5) {
      y = y + 1.0;
      x = x - 1;
  }
  if (isBalanced && (x < 10)) {
      y = y * 2;
  }
}
```

Question: Perform the Following Compiler Phases

1. **Lexical Analysis**

- Identify all **tokens** in the given source program.
- Write their corresponding **token names**, **lexemes**, and **patterns** (regular expressions).
  (Example: def → KEYWORD, x → IDENTIFIER, 10 → NUMBER, etc.)

2. **Syntax Analysis**

- Construct a simplified **Context-Free Grammar (CFG)** for Sci-PL that can parse:
  - Variable declarations (def type id;)
  - Assignment statements
  - if–else and while constructs
  - Arithmetic and logical expressions
  - check_balance(expr) function call
- Draw a **parse tree or abstract syntax tree (AST)** for the following statement:

  isBalanced = check_balance(r1);

3. **Intermediate Code Generation**

- Generate **Three-Address Code (TAC)** for the main operations, such as:

```
t1 = x + 5.5
y = t1
t2 = CALL check_balance(r1)
isBalanced = t2
if_false CALL check_balance(r2) goto L1
  t3 = x + 1
  x = t3
  goto L2
L1:
  t4 = x - 1
  x = t4
L2:
...
```

- Mark labels for control flow (if, else, while) and note how temporary variables are used.

4. Assembly Code Generation

- Generate Assembly Code of the **Three-Address Code (TAC)** for the main operations